FinAcumen: Financial Multimodal Reasoning via Self-Evolving Experience Memory Harness

Pianran Guo[1,*], Pengcheng Zhou[2,*], Yuchen Jian[1], Shuhua Chen[1], Zhongliang Yang[1], Linna Zhou[1]

[1]Beijing University of Posts and Telecommunications
[2]Queen Mary University of London
guopianran@bupt.edu.cn, jp2020213547@qmul.ac.uk
jianyucheng@bupt.edu.cn, lyonia_orangerica@bupt.edu.cn
yangzl@bupt.edu.cn, zhoulinna@bupt.edu.cn
[*]Equal contribution

Abstract

Financial multimodal reasoning requires agents to coordinate numerical computation, retrieval, visual interpretation, and temporal grounding across heterogeneous evidence sources. Existing tool-augmented agents improve execution fidelity, yet remain largely stateless across episodes, repeatedly rediscovering reasoning strategies and failure patterns. In high-stakes financial settings, this leads to unreliable tool routing, noisy retrieval, and hallucination-prone reasoning. We present FinAcumen, a financial reasoning agent framework centered on selective experience memory for tool-augmented multimodal reasoning. FinAcumen accumulates financially grounded reasoning experience from prior trajectories, distilling successful strategies and failure-derived cautionary rules into a persistent memory bank. During inference, retrieved experiences condition reasoning only when semantic relevance exceeds a calibrated threshold, while irrelevant memory is explicitly suppressed through a fallback mechanism. A deterministic financial tool environment further grounds numerical computation, retrieval, visual decoding, and answer verification. Across four financial multimodal reasoning benchmarks, FinAcumen consistently improves a frozen 8B vision-language model over finance-specialized models and approaches leading proprietary general-purpose models. Further analysis shows that selective experience activation improves reasoning reliability under retrieval uncertainty. Our code is available at https://github.com/CamelliaLilium/FinAcumen

1 Introduction

Financial multimodal reasoning requires agents to coordinate heterogeneous evidence from text, tables, charts, and time-indexed records. Unlike conventional question answering (QA), this setting jointly demands visual interpretation, numerical computation, retrieval, and temporal grounding across multiple interacting modalities. Appendix F presents representative items from the four benchmarks evaluated in this work. Recent benchmarks expose complementary aspects of this challenge, including quantity localization in filings of the United States Securities and Exchange Commission (SEC), chart-centric numerical reasoning, temporal-aware multimodal retrieval, and hallucination-sensitive financial analysis (Krumdick et al., 2024; Tang et al., 2025; Zhu et al., 2025; Luo et al., 2025).

Despite rapid progress in multimodal large language models (LLMs), reliable financial reasoning remains unresolved. General-purpose models lack financially grounded reasoning strategies under complex multimodal settings, while finance-specialized models often exhibit limited robustness outside their tuning distributions (Liu et al., 2025; Huang et al., 2024; Caillaut et al., 2025). Recent evidence shows that even models performing competitively on distribution-near tasks can fail sharply on chart-dense and retrieval-intensive settings, suggesting that static fine-tuning alone does not provide adaptive reasoning policies for heterogeneous modality compositions (Dai et al., 2026; Deng et al., 2026). Performance remains particularly constrained in high-difficulty settings, where benchmark leaders still exhibit substantial degradation under retrieval ambiguity, multi-step numerical reasoning, and hallucination-sensitive evaluation (Tang et al., 2026, 2025; Luo et al., 2025; Zhu et al., 2025).

In practical financial analysis workflows, human experts rely not only on domain knowledge, but also on accumulated experience about when to re-

Findings of the Association for Computational Linguistics: EMNLP 2026, pages 50675–50695
October 24-29, 2026 ©2026 Association for Computational Linguistics

trieve evidence, invoke tools, cross-check modalities, and verify intermediate conclusions. Tool-augmented paradigms such as ReAct externalize computation and perception, yet remain largely stateless across episodes and repeatedly rediscover reasoning procedures (Yao et al., 2023). Existing memory-based paradigms partially address this limitation by storing prior reflections or trajectories, but they often conflate successful and failed experiences, retrieve weakly related guidance, and expand without controlling retrieval quality (Shinn et al., 2023; Zhao et al., 2024; Ouyang et al., 2026; Zhang et al., 2026a). Such behavior is particularly problematic in finance, where irrelevant retrieval can directly degrade reasoning quality and hallucinated evidence may lead to incorrect analytical conclusions (Shi et al., 2023; Liu et al., 2024). Benchmarks such as FinTMMBench and FinMME make this issue explicit by penalizing retrieval errors and unsupported reasoning, requiring systems to determine not only what to retrieve, but also whether retrieved guidance should influence inference (Zhu et al., 2025; Luo et al., 2025).

We propose **FinAcumen**, a financial multimodal reasoning framework centered on selective experience-guided inference for tool-augmented agents. FinAcumen introduces a **Financial Memory (FM)** module that accumulates financially grounded reasoning experience from prior trajectories, separating reusable strategies from failure-derived cautionary rules. During inference, retrieved experience conditions reasoning only when semantic relevance exceeds a calibrated threshold; otherwise, the model explicitly falls back to its base reasoning policy to suppress noisy memory injection. A complementary **Financial Tools (FT)** environment provides deterministic support for numerical computation, retrieval, visual decoding, and answer verification.

Our contributions are summarized as follows:

1. We introduce a selective experience memory framework for financial multimodal agents that separates reusable reasoning strategies from failure-derived cautionary guidance.

2. We propose a retrieval-conditioned inference mechanism in which memory activates only under sufficient semantic relevance, enabling fallback under retrieval uncertainty.

3. Across four financial multimodal reasoning benchmarks, FinAcumen consistently improves its frozen 8B vision-language model (VLM), exceeds the reported finance-specialized baselines on shared settings, and remains competitive with leading proprietary models.

2 Related Work

Financial multimodal reasoning. Financial QA evaluation has expanded from text-only settings to multimodal, temporal-aware, and hallucination-sensitive benchmarks. Representative datasets cover SEC-grounded quantity localization (BizBench, where we evaluate the SEC-NUM subset), chart-centric numerical reasoning (FinMMR), temporal multimodal retrieval and reasoning (FinTMMBench), and broad financial multimodal assessment (FinMME) (Krumdick et al., 2024; Tang et al., 2025; Zhu et al., 2025; Luo et al., 2025). On the modeling side, finance-oriented LLMs such as Fin-R1 (Liu et al., 2025), Open-FinLLMs (Huang et al., 2024), and LLM Pro Finance Suite (Caillaut et al., 2025) mainly inject domain knowledge through parameter updates. In contrast, our formulation in §3 and §4 keeps the backbone frozen and adapts at inference time through a fixed tool suite and retrieved experience, so performance differences are attributable to experience conditioning rather than model re-training.

Cross-episode experience memory. Research on cross-episode agent improvement can be viewed along three steps: memory construction, retrieval decision, and bank maintenance. ReAct (Yao et al., 2023) established tool-interleaved reasoning but remains episode-local. Reflexion (Shinn et al., 2023), ExpeL (Zhao et al., 2024), ReasoningBank (Ouyang et al., 2026), and EchoSafe (Zhang et al., 2026a) move to persistent experience accumulation across episodes. For retrieval, prior evidence shows that irrelevant context can harm reasoning quality (Shi et al., 2023; Liu et al., 2024), while Self-RAG (Asai et al., 2024) highlights explicit retrieve-or-not decisions.

Recent work also explores task-specific memory and skill artifacts. M* evolves task-specific memory programs, whereas ASDA distills structured skills for financial reasoning without modifying model weights (Pan et al., 2026; Yim et al., 2026).

The use of a bank constructed before evaluation also follows established offline protocols. ExpeL distills insights from training tasks for use on unseen tasks (Zhao et al., 2024); Agent Work-

flow Memory induces reusable workflows offline from training trajectories (Wang et al., 2025); and MemGen constructs generative memory offline and keeps the memory pathway fixed during inference (Zhang et al., 2026b). Relative to the cited approaches, FinAcumen combines three design choices: structured Findings and Cautions, an annotator that converts retrieved experience into directives for the current query, and calibrated thresholding with an empty-return fallback under a fixed tool suite.

3 Problem Formulation

Notation. We consider multimodal financial QA with disjoint train and test splits. Each instance $x = (q, c, I)$ comprises question q, textual context c, and optional images I, with gold answer $y^\star(x)$ scored under benchmark metric μ. A frozen VLM π_θ performs multi-turn tool-augmented decoding over a fixed tool suite $\mathcal{T}$ (§4.4). A memory bank $\mathcal{M}$, populated from training trajectories, stores structured experience entries. At inference, entries are retrieved via embedding similarity and the top-k are injected into the prompt (Algorithm 1); a complete symbol table appears in Appendix A.

Given $\mathcal{M}$ populated from $\mathcal{D}_{\text{train}}$ trajectories, we seek $\mathcal{M}^\star$ that maximizes expected accuracy on $\mathcal{D}_{\text{train}}$:

$$\mathcal{M}^\star \in \arg\max_{\mathcal{M}} \; \mathbb{E}_{x\sim\mathcal{D}_{\text{train}}}\Big[\, \mu\big(\pi_\theta(x; \mathcal{M}, \mathcal{T}),\, y^\star(x)\big)\Big]. \tag{1}$$

At test time, $\mathcal{M}^\star$ is frozen and read-only: queries may activate stored patterns but never write to the bank. Freezing the bank removes a potential leakage path in which gold-answer feedback from one test item is written to memory and later retrieved by another, thereby preserving the intended independence and reproducibility of held-out evaluation. Final performance is reported as $\mathbb{E}_{x\sim\mathcal{D}_{\text{test}}}[\mu(\pi_\theta(x; \mathcal{M}^\star, \mathcal{T}), y^\star(x))]$.

The discrete optimization in (1) is approximated in Section 4 through a writer-retriever loop (Algorithm 1) that constructs $\mathcal{M}$ from graded training trajectories.

4 Method

4.1 Overview

As defined in Section 3, a frozen language model π_θ operates over a tool suite $\mathcal{T}$ to solve financial problems. FINACUMEN extends this formulation with two components, illustrated in Figure 1: Financial Memory (FM) and Financial Tools (FT). FM enables the model to accumulate experience from its own solution trajectories and retrieve it as guidance for new problems, growing a bank of generalized strategies and guard rules through multi-trajectory sampling. FT provides a deterministic execution layer that offloads computation, data retrieval, visual decoding, and answer consolidation from the generative process. The pipeline is shared across accumulation and inference, isolating memory retrieval as the sole conditioned pathway.

4.2 Financial Memory

Memory bank structure. Each entry in $\mathcal{M}$ is a structured record of one completed problem, containing the original question, its gold answer, and the distilled experience. The experience consists of two parts: strategies generalized from trajectories that reached the correct answer, and guard rules extracted from those that did not.

Memory consolidation. For each problem the model encounters, the framework generates multiple independent solution trajectories from π_θ with diverse decoding, scores each against the gold answer, and feeds the scored set to a single summary agent. Let $\{\text{Traj}_1, \ldots, \text{Traj}_K\}$ denote the trajectories for problem x with gold answer y^*, and let $s_k = \text{Score}(\text{Traj}_k, y^*)$ be the correctness score assigned to each. The summary agent synthesizes two outputs:

$$(S, C) = \text{Summarize}\big(\{(\text{Traj}_k, s_k)\}_{k=1}^{K},\; y^*\big), \tag{2}$$

where S collects strategies from successful trajectories and C collects cautions from failed ones. These are written into a new entry alongside the original question and answer, incrementally expanding the bank for future retrieval.

Memory activation. At inference, a query x is embedded into the same semantic space as the bank through the shared encoder $E(\cdot)$. Its similarity to each entry m is defined by cosine distance over the resulting vectors:

$$\text{sim}(x, m) = \frac{E(x) \cdot z_m}{\|E(x)\|\, \|z_m\|}, \tag{3}$$

where z_m denotes the pre-stored embedding of entry m. Entries passing a calibrated threshold τ form the candidate set

$$\mathcal{M}_x = \{\, m \in \mathcal{M} \mid \text{sim}(x, m) \geq \tau \,\}, \tag{4}$$

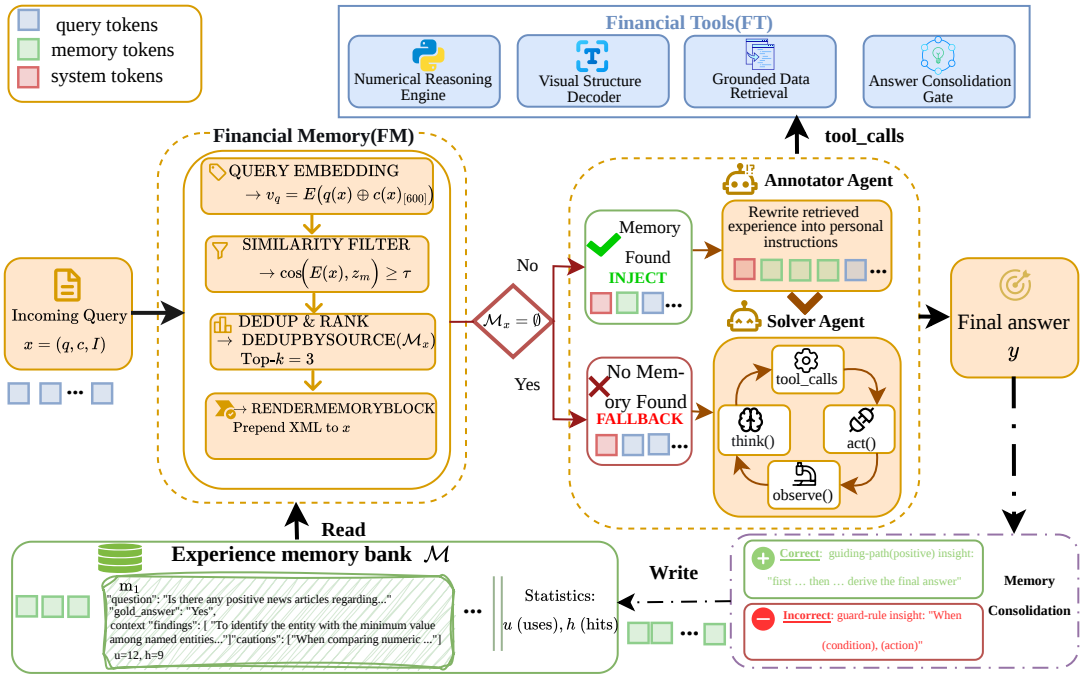


Figure 1: The FINACUMEN pipeline. Experience accumulated through multi-trajectory sampling is stored in FM as generalized strategies and guard rules. At inference, semantically similar entries are retrieved and provided as in-context guidance; when none qualify, the model proceeds with FT alone.

which is then deduplicated and ranked. The top-ranked subset $\mathcal{M}_x^*$ is rendered as a structured prefix in Extensible Markup Language (XML) and prepended before the problem prompt. The model produces its final answer y through a tool-augmented rollout

$$y = \begin{cases} \text{Rollout}\big(\pi_\theta,\ x_{\text{mem}},\ \mathcal{T}\big), & \mathcal{M}_x^* \neq \emptyset \\ \text{Rollout}\big(\pi_\theta,\ x,\ \mathcal{T}\big), & \text{otherwise}, \end{cases} \tag{5}$$

where x_{mem} denotes the prompt with memory prefix prepended.

4.3 Agent Architecture

All LLM-driven roles in FINACUMEN share the same base model and are distinguished by their prompt-defined behavior. During consolidation, the model acts as a sampling agent that generates multiple diverse solution trajectories, and as a summary agent that scores them against the gold answer before synthesizing strategies from successes and cautions from failures. During inference, an annotator agent rewrites retrieved experiences into directives personalized to the current problem before the solver agent plans and executes tool-augmented reasoning. The solver internally decomposes into two phases: planning, where it inventories inputs, judges experience applicability, and formulates a strategy; and execution, where it invokes tools and terminates with the final answer.

4.4 Financial Tools

The tool suite $\mathcal{T}$ comprises four tools:

$$\mathcal{T} = \left\{ \begin{array}{l} \text{Numerical Reasoning Engine}, \\ \text{Grounded Data Retrieval}, \\ \text{Visual Structure Decoder}, \\ \text{Answer Consolidation Gate} \end{array} \right\}, \tag{6}$$

covering precise arithmetic, factual data lookup, chart interpretation, and answer verification. The same tool set and calling schema are shared across all configurations, so that tool mechanics remain identical whether or not FM conditions the prompt.

Numerical Reasoning Engine. This tool decouples financial arithmetic from the generative process by executing code in a persistent namespace. Let σ_t denote the variable bindings accumulated across consecutive calls within the same agent loop, with σ_0 containing no bindings:

$$(o_t, \sigma_t) = \text{EXECUTE}(\text{code}_t,\ \sigma_{t-1}), \tag{7}$$

where o_t is the captured output. This stateful design enables cascaded operations such as rate compounding and cross-statement aggregation that stateless code generators cannot reliably chain: without persistent state, the model must reinitialize all quantities in every code block, risking inconsistency across arithmetic steps. Financial multi-step reasoning chains intermediate quantities by nature: growth rates build on prior period values, ratios depend on previously computed numerators and denominators. A fresh execution session per code call severs this dependency chain; the persistent namespace preserves it.

Grounded Data Retrieval. This tool resolves factual financial queries against a curated panel of open, high, low, and close (OHLC) price records, news corpora, and quarterly indicator tables. Given a symbol, date, and indicator, the lookup returns the corresponding structured entry:

$$r = \textsc{Lookup}(\text{symbol},\ \text{date},\ \text{indicator}), \quad (8)$$

anchoring each numerical claim to a verifiable source row. All filtering logic executes deterministically within the tool backend, preventing the model from fabricating lookup parameters or results. Financial quantities such as quarterly revenue or daily closing prices are factual, time-bound values. Parametric knowledge, encoded during pretraining, cannot reliably reproduce them when the query demands a specific date or indicator; consulting a deterministic panel anchors each claim in an externally verifiable source.

Visual Structure Decoder. This tool handles chart and table images in two layers. Before the agent loop begins, all available images are transcribed through optical character recognition (OCR) and prepended to the context as structured text, providing precise numerical inputs upfront. During reasoning, if the agent detects that computation results contradict values previously read from images, it may explicitly invoke this tool to re-parse the source image, producing an updated structured representation that resolves the discrepancy.

Answer Consolidation Gate. This tool serves as the sole termination point of the agent loop. Given the accumulated trajectory Traj and the proposed answer a, it enforces three conditions before emission:

$$\begin{aligned}\textsc{Verify}(\mathrm{Traj}, a) \ \equiv\ & \text{source-traceable}(\mathrm{Traj}) \\ & \wedge\ \text{units-ok}(\mathrm{Traj}) \\ & \wedge\ \text{format-valid}(a),\end{aligned} \quad (9)$$

where source-traceable checks that a or its numeric equivalent appears among the captured outputs of Execute calls, the returned entries of Lookup calls, or the OCR-extracted values in the context; units-ok verifies that quantity units remain compatible across tool calls without unresolved conversions; and format-valid ensures a conforms to the task instruction. Because termination is already the final reasoning step, embedding the verification check there costs no additional inference; a separate post-hoc auditor would require a second traversal of the full trajectory.

Interface with FM. FM is the memory bank described in §4.2; π_θ is the frozen language model. FM operates strictly at the prompt level and never interleaves with the tool-calling mechanics. The baseline executes $\textsc{Rollout}(\pi_\theta, x, \mathcal{T})$ without memory, while the memory-active path prepends retrieved experiences before x. Because $\mathcal{T}$ and the calling protocol remain identical across both paths, any performance difference is attributable solely to the experiential guidance rather than altered tool dynamics.

5 Experiments

5.1 Experimental Setup

Benchmarks. We evaluate six settings from four benchmarks, covering numerical grounding, multimodal numerical reasoning, temporal-aware retrieval, and chart understanding. BizBench SEC-NUM (Krumdick et al., 2024) includes 2000 SEC 10-K/10-Q items for quantity localization under numeric distractors; we use exact match with 1% relative tolerance. FinMMR (Tang et al., 2025) includes Easy/Medium/Hard splits (1200/1200/1000) for bilingual multi-image financial numerical reasoning; we follow its 0.2% relative tolerance criterion. FinTMMBench (Zhu et al., 2025) includes 1136 temporal-aware multimodal retrieval-reasoning items over tables, news, prices, and charts; we report official GPT-4o-mini-judge accuracy. FinMME (Luo et al., 2025) includes 2220 chart-intensive items across diverse financial domains. We report Avg as the arithmetic mean of

the six dimensions defined by the benchmark. Because the released annotations do not provide official per-sample domain labels, we reconstruct the corresponding dimension partition as documented in Appendix G.1. We follow each benchmark's official split and evaluation protocol. Full dataset, scoring, and implementation details are provided in Appendix G. Representative benchmark items are provided in Appendix F.

Implementation details. We use frozen Qwen3-VL-8B-Instruct (Bai et al., 2025) through the DashScope application programming interface (API) with temperature 0. Without augmentation, it serves as the BASE model-only baseline in Table 1. As detailed in Section 4.4, the tool suite (BASE + FT) equips the backbone with a ReAct loop (max 16 steps) over four tool components for numerical reasoning, data retrieval, visual decoding, and answer gating; its standalone contribution is isolated in the ablation study (Section 5.3). The FM bank pools ∼2,400 experience accumulation trajectories across all benchmarks, sourced exclusively from training/validation splits held out from test data, and is frozen read-only at evaluation. All ablation conditions share identical tool and decoding configurations, isolating the effect of FM and FT. Published baseline numbers are collected from respective benchmark sources; exact model variant specifications and per-cell attributions appear in Appendix G.3, with further implementation details in Appendix G. Construction and inference cost estimates appear in Appendix K.

5.2 Main Results

Table 1 presents a system-level comparison under a constrained deployment setting. In this work, constrained deployment refers to inference-time augmentation of a selected frozen backbone with external memory and deterministic tools while preserving the backbone parameters and deployment interface. The selected backbone may be accessed through an API or deployed locally. Published closed-source baselines serve as benchmark-reported capability references and do not use FM or FT. Accordingly, the controlled comparison is between Base and FINACUMEN under the same backbone and evaluation protocol.

The difference between Base and FINACUMEN estimates the framework contribution under the shared backbone, while the best reported General LLM result indicates the remaining capability gap under each benchmark setting. Model variant specifics are documented in Appendix G.3.

Table 1 supports three observations. **First, FINACUMEN produces large system-level gains**: the same frozen backbone rises from 27.67 to 68.65 on BizBench and from 27.84 to 51.08 on the FinMME Avg metric, while also surpassing the published GPT-4o entries on FinMMR Easy/Medium. **Second, it is more robust than static financial specialization**: across all shared columns, FINACUMEN outperforms Llama-Open-Finance-8B, FinLLaVA, and Fin-R1, suggesting that financial multimodal QA benefits from test-time evidence routing, verification, and strategy adaptation. **Third, the remaining gaps are informative**: FINACUMEN still trails the best General LLM on BizBench, FinMMR Hard, and FinTMMBench, where stronger planning, temporal retrieval, or difficult numerical reasoning is required. Thus, FINACUMEN is best viewed as a deployment-efficient way to strengthen frozen VLMs, not as evidence that an 8B backbone has superior intrinsic parametric ability. Repeated-run uncertainty, experience-memory baseline comparisons, and cross-backbone evaluations are reported in Appendices H, I, and J.

5.3 Ablation Study

Since Table 1 compares a full system against published model-only baselines, we isolate the source of the gain by toggling FT and FM on the same Qwen3-VL-8B backbone. Table 2 reports FinMMR Easy, whose moderate base accuracy (59.17) makes module-level gains easy to measure. All variants share the same backbone, decoding configuration, and scoring protocol.

The ablation explains the main-table gains. Adding FT raises accuracy from 59.17 to 74.08, showing that tools resolve many execution-level errors. Adding FM on top of the same tool suite further improves accuracy to 81.67, a 7.59-point gain over tool-only inference. This shows that FINACUMEN is not merely a tool wrapper: retrieved experiences provide strategy-level guidance about tool use, quantity verification, and recurring financial reasoning traps. Together, FT and FM yield a 22.50-point improvement over the base model. The larger-backbone component study in Appendix J shows a similar complementary pattern.

5.4 Memory Accumulation Behavior

To isolate the marginal contribution of experience memory, we evaluate a frozen FM bank on

Table 1: Per-benchmark accuracy across six financial QA settings. Published baseline entries are benchmark-reported model results without our FM and FT wrapper; per-cell variant details and source attributions are in Appendix G.3. Best result per column in bold, second-best underlined. A dash indicates unreported by the source benchmark. FINACUMEN results use a frozen Qwen3-VL-8B-Instruct backbone with FM and FT.

Group	Model	BizBench	FinMMR			FinTMMBench	FinMME
		SEC-NUM	Easy	Medium	Hard		
General LLMs	GPT	**79.30**	78.00	63.33	45.40	**21.53**	46.56
	Claude Sonnet	63.39	77.00	62.25	**50.80**	–	48.20
	Qwen2.5-VL-72B	69.85	77.42	63.42	43.30	15.15	**52.54**
	Llama	74.70	77.83	63.25	48.70	11.57	–
	Gemini 2.0 Flash	–	74.92	57.83	44.40	18.42	51.85
Financial LLMs	Llama-Open-Finance-8B	63.64	–	–	–	–	29.95
	FinLLaVA	53.46	3.25	2.75	3.90	11.80	27.70
	Fin-R1	59.32	–	–	–	–	26.62
Base	Qwen3-VL-8B-Instruct	27.67	59.17	45.75	31.40	16.20	27.84
FINACUMEN	Qwen3-VL-8B + FINACUMEN	68.65	**81.67**	**67.00**	46.10	19.98	51.08

Table 2: Ablation of FT and FM on FinMMR Easy with Qwen3-VL-8B-Instruct. Δ denotes absolute improvement over the Base only variant.

Variant	Accuracy	Δ
Base only	59.17	–
Base + FT	74.08	+14.91
Base + FT + FM (full FINACUMEN)	81.67	+22.50

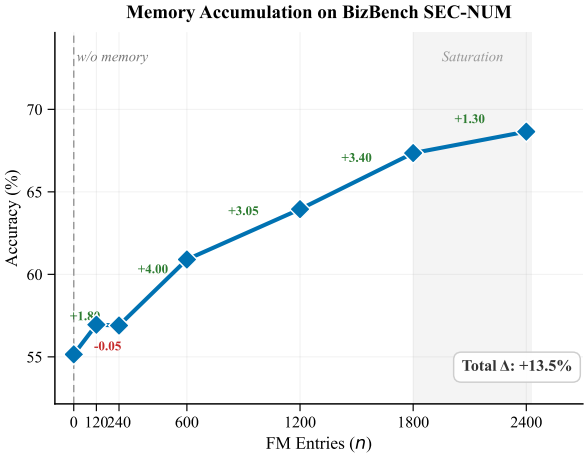


Figure 2: Accuracy on BizBench SEC-NUM as a function of accumulated FM bank size. The $n = 0$ condition includes the FT tool suite. Per-step delta annotations reveal diminishing marginal returns, with accuracy approaching a saturation regime beyond $n = 1800$, marked by the shaded region.

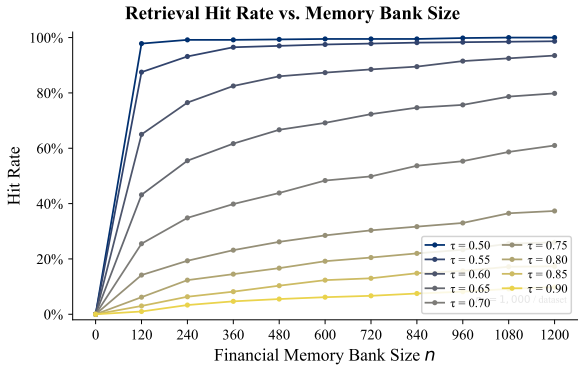
BizBench SEC-NUM across seven snapshots spanning 0 to 2400 accumulated entries. The $n = 0$ condition corresponds to the FT-augmented agent with an empty FM bank and is distinct from the bare model baseline.

Figure 2 reports the accuracy trajectory. Starting from 55.15% with no experience entries, accuracy rises to 68.65% at 2400 entries, a net gain of 13.5 points. The curve exhibits diminishing returns: per-interval gains are most pronounced at smaller bank sizes and progressively decline, entering a saturation regime beyond 1800 entries where additional entries produce marginal improvements. A detailed decomposition of per-interval marginal gains is provided in Appendix C. Across the evaluated snapshots, accuracy increases overall as the bank expands to 2400 entries, while the smaller gains at larger bank sizes indicate diminishing marginal returns. This analysis characterizes the relationship between bank size and test accuracy under the fixed retrieval setting; it does not by itself isolate the effect of selective retrieval.

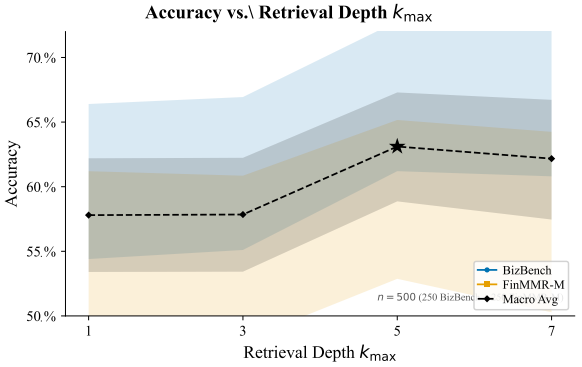
5.5 Sensitivity Analysis

The FM retrieval pipeline has two hyperparameters: a similarity threshold τ governing entry admission and a retrieval depth $k_{\max}$ governing how many admitted entries enter the reasoning context. We analyze each independently.

Cosine similarity threshold τ. We evaluate τ via hit rate rather than downstream accuracy, as this threshold must establish a viable candidate pool before $k_{\max}$ can operate; hit rate measures the fraction of test queries for which at least one bank entry exceeds the threshold. Figure 3a shows hit rates for nine τ values spanning $[0.50, 0.90]$ against accumulated bank size. Three operational regimes emerge. A loose-filtering regime at $\tau \leq 0.55$ exceeds 87% hit rate at 120 entries with negligible semantic filtering. A strict-blocking regime at $\tau \geq 0.75$ stays



(a) Hit rate as a function of bank size, stratified by similarity threshold τ.



(b) Accuracy as a function of retrieval depth $k_{\max}$.

Figure 3: Sensitivity analysis of two retrieval hyperparameters. (a) Hit rate across bank sizes for different cosine thresholds; three operational regimes emerge, with the practical range centered at moderate filtering strictness. (b) Accuracy as a function of retrieval depth, following an inverted-U pattern that peaks at moderate depth and degrades at both extremes. Shaded regions denote 95% bootstrap confidence intervals.

below 37% even at 1200 entries, denying most queries access to memory. A moderate-filtering regime in $[0.60, 0.70]$ yields 43% to 80% across bank expansion. At $\tau = 0.65$, the center of this moderate regime, hit rate rises from 43.2% at 120 entries to 79.8% at 1200 entries; saturation beyond 600 entries mirrors the accumulation plateau observed in Figure 2, consistent with early entries capturing prevalent error patterns. We therefore select $\tau = 0.65$ as a balance between retrieval coverage and filtering strength and keep this value fixed across benchmarks to avoid tuning for individual benchmarks.

Retrieval depth $k_{\max}$. We test $k_{\max} \in \{1, 3, 5, 7\}$ at $\tau = 0.65$ on 500 questions evenly split across BizBench and FinMMR-M, with all other configurations matching the main experiments. Figure 3b shows an inverted-U relationship between retrieval depth and accuracy. At $k_{\max} = 1$ macro accuracy is 57.80%; expanding to 3 reaches 57.85%, a shift of 0.05 percentage points that lies within bootstrap confidence intervals and is thus not statistically meaningful. At $k_{\max} = 5$ macro accuracy rises to 63.11%, a gain of 5.26 points that falls well outside the confidence bounds, with consistent improvements on BizBench of 5.98 points and FinMMR-M of 4.55 points. This sharp transition is consistent with a scenario where entries ranked fourth and fifth cover strategy categories absent from the top three; the highest-ranked entries tend to cluster around a single dominant error type, while moderately-ranked entries span complementary failure modes demanding qualitatively different solution patterns. At $k_{\max} = 7$ macro accuracy drops to 62.17%, a decline of 0.94 points, indicating that retrieval beyond five entries introduces distractors that outweigh any additional strategic signal. The tension between insufficient strategic coverage at low $k_{\max}$ and signal dilution at high $k_{\max}$ is resolved at $k_{\max} = 5$, which is adopted for all experiments.

6 Conclusion

We introduced FINACUMEN, a framework coupling self-evolving Financial Memory with deterministic Financial Tools for financial multimodal QA. Across four benchmarks, FINACUMEN improves its frozen 8B backbone, exceeds the reported finance-specialized baselines on shared settings, and remains competitive with leading general-purpose systems. Ablation confirms complementary roles: FM improves routing and strategy selection via polarity-aware memory activation, while FT guarantees numerical fidelity through execution-environment grounding; when no relevant experience is retrievable, FM falls back to the tool-augmented baseline without degradation. The decoupled architecture, in which an evolving experience bank operates orthogonally to a deterministic tool suite around a frozen base model, suggests a general paradigm for artificial intelligence systems in professional domains: externalized memory captures reusable strategy patterns without destabilizing the core model, while domain-specific tools provide computational guarantees that generative

components alone typically struggle to guarantee. Future work includes sample-efficient consolidation and compositional memory to reduce cold-start cost and support cross-task transfer, together with cross-lingual alignment and stronger visual encoders for multilingual and chart-dense deployment settings.

7 Limitations

We acknowledge two primary limitations. First, Financial Memory construction requires auxiliary LLM calls for trajectory tagging, embedding, and experience consolidation, introducing a non-trivial upfront cost during bank initialization. Although retrieval remains efficient after consolidation, improving sample efficiency and enabling incremental online accumulation remain important future directions. Second, our evaluation is limited to English financial multimodal benchmarks. Financial documents vary substantially across languages in terminology, tabular conventions, and chart structures, which may limit direct transferability of retrieved experience and tool configurations. Extending experience-conditioned reasoning to multilingual financial settings and broader model families remains future work.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62172053 and Grant 62302059.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, and 45 others. 2025. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*.

Gaëtan Caillaut, Raheel Qader, Jingshu Liu, Mariam Nakhlé, Arezki Sadoune, Massinissa Ahmim, and Jean-Gabriel Barthelemy. 2025. The LLM pro finance suite: Multilingual large language models for financial applications. *arXiv preprint arXiv:2511.08621*.

Yuyang Dai, Yan Lin, Zhuohan Xie, and Yuxia Wang. 2026. RealFin: How well do LLMs reason about finance when users leave things unsaid? In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 25050–25080, San Diego, California, United States. Association for Computational Linguistics.

Shuangyan Deng, Zhongsheng Wang, Rui Mao, Ciprian Doru Giurcăneanu, and Jiamou Liu. 2026. CLER: Improving multimodal financial reasoning by cross-MLLM error reflection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30494–30502.

Jimin Huang, Mengxi Xiao, Dong Li, Zihao Jiang, Yuzhe Yang, Yifei Zhang, Lingfei Qian, Yan Wang, Xueqing Peng, Yang Ren, Ruoyu Xiang, Zhengyu Chen, Xiao Zhang, Yueru He, Weiguang Han, Shunian Chen, Lihang Shen, Daniel Kim, Yangyang Yu, and 25 others. 2024. Open-FinLLMs: Open multimodal large language models for financial applications. *arXiv preprint arXiv:2408.11878*.

Michael Krumdick, Rik Koncel-Kedziorski, Viet Dac Lai, Varshini Reddy, Charles Lovering, and Chris Tanner. 2024. BizBench: A quantitative reasoning benchmark for business and finance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8309–8332, Bangkok, Thailand. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Zhaowei Liu, Xin Guo, Zhi Yang, Fangqi Lou, Lingfeng Zeng, Jinyi Niu, Mengping Li, Qi Qi, Zhiqiang Liu, Yiyang Han, Dongpo Cheng, Ronghao Chen, Huacan Wang, Xingdong Feng, Huixia Judy Wang, Chengchun Shi, and Liwen Zhang. 2025. Fin-R1: A large language model for financial reasoning through reinforcement learning. *arXiv preprint arXiv:2503.16252*.

Junyu Luo, Zhizhuo Kou, Liming Yang, Xiao Luo, Jinsheng Huang, Zhiping Xiao, Jingshu Peng, Chengzhong Liu, Jiaming Ji, Xuanzhe Liu, Sirui Han, Ming Zhang, and Yike Guo. 2025. FinMME: Benchmark dataset for financial multi-modal reasoning evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29465–29489, Vienna, Austria. Association for Computational Linguistics.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*.

Wenbo Pan, Shujie Liu, Xiangyang Zhou, Shiwei Zhang, Wanlu Shi, Mirror Xu, and Xiaohua Jia. 2026. M*: Every task deserves its own memory harness. *arXiv preprint arXiv:2604.11811*.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 31210–31227. PMLR.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652.

Zichen Tang, Haihong E, Rongjin Li, Jiacheng Liu, Linwei Jia, Zhuodi Hao, Zhongjun Yang, Yuanze Li, Haolin Tian, Xinyi Hu, Peizhi Zhao, Yuan Liu, Zhengyu Wang, Xianghe Wang, Yiling Huang, Xueyuan Lin, Ruofei Bai, Zijian Xie, Qian Huang, and 2 others. 2026. FinMMDocR: Benchmarking financial multimodal reasoning with scenario awareness, document understanding, and multi-step computation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 25858–25866.

Zichen Tang, Haihong E, Jiacheng Liu, Zhongjun Yang, Rongjin Li, Zihua Rong, Haoyang He, Zhuodi Hao, Xinyang Hu, Kun Ji, Ziyan Ma, Mengyuan Ji, Jun Zhang, Chenghao Ma, Qianhe Zheng, Yang Liu, Yiling Huang, Xinyi Hu, Qing Huang, and 2 others. 2025. FinMMR: Make financial numerical reasoning more multimodal, comprehensive, and challenging. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3245–3257.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Tik Yu Yim, Wenting Tan, Sum Yee Chan, Tak-Wah Lam, and Siu Ming Yiu. 2026. ASDA: Automated skill distillation and adaptation for financial reasoning. *arXiv preprint arXiv:2603.16112*.

Ce Zhang, Jinxi He, Junyi He, Katia Sycara, and Yaqi Xie. 2026a. Evolving contextual safety in multimodal large language models via inference-time self-reflective memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 41182–41192.

Guibin Zhang, Muxin Fu, and Shuicheng Yan. 2026b. MemGen: Weaving generative latent memory for self-evolving agents. In *The Fourteenth International Conference on Learning Representations*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.

Fengbin Zhu, Junfeng Li, Liangming Pan, Wenjie Wang, Fuli Feng, Chao Wang, Huanbo Luan, and Tat-Seng Chua. 2025. Towards temporal-aware multi-modal retrieval augemented generation in finance. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 6289–6297.

Appendix

A Notation 11
B Inference Algorithm 11
C Marginal Gain Decomposition of Memory Accumulation 11
D Memory Strategy Ablation 12
E Retrieval Stability Analysis 13
E.1 Experimental Design 13
E.2 Retrieval Metrics 13
E.3 Coverage and Retrieval Density 13
E.4 Retrieval Stability 14
F Benchmark Examples 15
G Experimental Details 15
G.1 Evaluation Protocols 15
G.2 FinAcumen Configuration 15
G.3 Baseline Attribution 16
H Repeated-Run Uncertainty 17
I Experience-Memory Baselines 17
J Cross-Backbone and Scale Generalization 17
K Cost Analysis 18
L Prompt Templates 19

A Notation

We define mathematical symbols that appear across multiple sections of the main text. Symbols introduced and used only within a single paragraph or algorithm are defined locally and not repeated here.

Table 3: Global mathematical notation.

Symbol	Definition
Problem and model	
$x = (q, c, I)$	Instance: question q, text context c, optional images I
π_θ	Frozen vision-language model with parameters θ
$\mathcal{T}$	Financial tool suite (§4.4)
Memory bank and retrieval	
$\mathcal{M}$	Experience memory bank
z_m	Pre-computed embedding of entry $m \in \mathcal{M}$
$E(\cdot)$	Unit-scale text embedding operator
τ	Cosine similarity threshold ($\tau = 0.65$)

B Inference Algorithm

Algorithm 1 FINACUMEN single-instance inference with retrieval

Require: x, $\mathcal{M}$, π_θ, $\mathcal{T}$, ω, E_{mgr}
$v_q \leftarrow E(q(x) \oplus c(x)_{[:600]})$
$V_{\mathcal{M}} \leftarrow E_{\text{mgr}}.\text{resolve_all}(\mathcal{M})$
$\vec{s} \leftarrow v_q \cdot V_{\mathcal{M}}^{\top}$
$I \leftarrow \{i \mid s_i \geq 0.65\}$
if $I = \emptyset$ **then**
  return ROLLOUT$(\pi_\theta, x, \mathcal{T})$
end if
$\vec{I} \leftarrow$ DESCENDINGSORT$(I, \vec{s})$
$\vec{I} \leftarrow$ DEDUPBYSOURCE$(\vec{I})$
$\vec{I}_{\text{top}} \leftarrow \vec{I}_{[:5]}$
xml $\leftarrow$ RENDERMEMORYBLOCK$(\mathcal{M}[\vec{I}_{\text{top}}])$
$y \leftarrow$ ROLLOUT$\big(\pi_\theta,$ PREPEND$(\text{xml}, x), \mathcal{T}\big)$
UPDATESTATS$(\vec{I}_{\text{top}}, y)$
if ω **then**
  schedule async CONSOLIDATE$(x, y, \mathcal{M})$
end if
return y

C Marginal Gain Decomposition of Memory Accumulation

Figure 4 decomposes the memory accumulation curve from Section 5.4 into per-segment marginal gains, defined as the accuracy improvement per 100 accumulated FM entries for each consecutive snapshot interval. The evaluation protocol mirrors the main analysis: the FT-augmented Qwen3-VL-8B-Instruct backbone is evaluated on the full 2000-item BizBench SEC-NUM test set at seven bank snapshots spanning 0 to 2400 entries.

The marginal gain decomposition in Figure 4 reveals a three-phase accumulation dynamic on BizBench. **Phase 1 (0–120 entries):** The initial accumulation of 120 entries yields a marginal gain of +1.50 per 100 entries, capturing the most common error patterns. **Phase 2 (240–1800 entries):** The next 1560 entries sustain a robust marginal efficiency of +0.51 to +1.11 per 100 entries, with the 240–600 interval producing the strongest per-entry return (+1.11) and the 600–1200 interval yielding +0.51. The 1200–1800 interval yields +0.57 per 100 entries, reflecting an uptick in marginal efficiency as newly accumulated entries target previously uncovered error patterns. **Phase 3 ($n \geq 1800$):** Beyond 1800 entries, the marginal gain declines to +0.22 per 100 entries, or approximately +1.30 points across the final 600-entry block. This diminishing return supports the selection of 2400 entries as the saturation knee for the production bank.

The consistently positive marginal gains across all intervals, including the saturation phase, con-

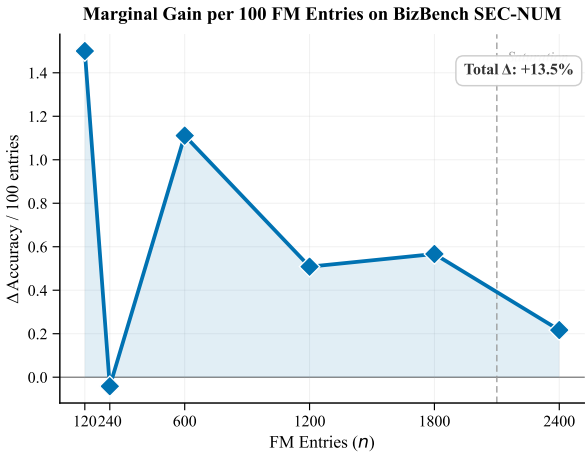


Figure 4: Marginal accuracy gain per 100 accumulated FM entries on BizBench SEC-NUM. The curve traces a diminishing-returns pattern across consecutive snapshot intervals. The dashed vertical line separates the main accumulation phase from the saturation regime.

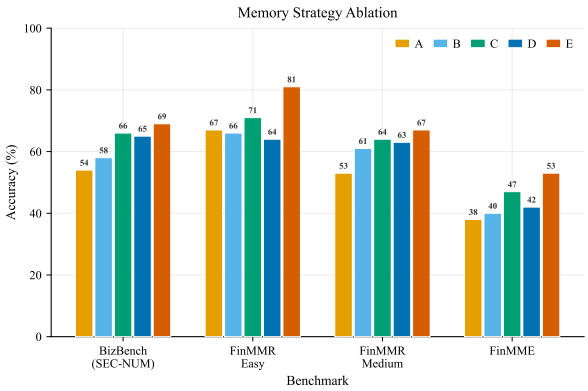


Figure 5: Memory strategy ablation on a shared test subsample. Checkmarks in Table 4 indicate the injected fields for each strategy.

firm that the FM bank continues to provide useful signal even after substantial accumulation. While individual snapshots contain minor fluctuations (the 120–240 interval shows a near-zero change of -0.04), the overall trajectory is monotonically increasing on the macro timescale. This characteristic supports our deployment strategy of accumulating experience passively throughout training and freezing the bank at the empirically determined saturation point for test-time inference, as described in Section 5.4.

D Memory Strategy Ablation

This appendix isolates the marginal contribution of each component in the FINACUMEN pipeline by comparing five injection strategies that toggle four fields of the retrieved memory block.

- **Q** (Question): the original question text of the top-retrieved similar item.
- **A** (Answer): the gold answer of the retrieved item.
- **E** (Experience): distilled Findings, reusable solution recipes, and Cautions, guard rules from past failures.
- **Annot** (Annotation): LLM-generated personalized directives that map generic rules to query-specific entities such as ticker symbols, date ranges, and metric names.

All strategies share Qwen3-VL-8B-Instruct as the backbone and operate on the same 600-question test subsample with MIN_COSINE set to 0.60. Strategies A and B use the single-shot baseline-raw variant without tools; strategies C through E use the full FinAcumen variant with the four-tool ReAct loop. Table 4 reports per-benchmark accuracy and the macro average across the four benchmarks.

The strategy comparisons in Table 4 decompose the contribution of each field. Adding gold answers to Q-only references, A to B, yields 3.25 macro points, confirming that reference answers carry a measurable signal. Adding tools and annotation without experience, B to D, yields 2.25 points, less than half the gain from experience alone.

Experience provides the dominant marginal improvement. Strategy C, which includes Q, A, and experience but omits annotation, exceeds strategy D, which includes Q, A, and annotation but omits experience, by 3.50 macro points. This gap isolates Findings and Cautions from the effect of annotation. The gain is concentrated on FinMME, from 42 to 47, and on FinMMR Easy, from 64 to 71, benchmarks on which sign convention and unit conversion errors recur systematically and are captured by distilled guard rules.

Annotation amplifies experience rather than adding independently to it. Strategy E, the full pipeline, exceeds C by 5.50 points, a margin larger than the D-to-C gap of 3.50. This asymmetry indicates that annotator directives are most effective when experience content is already rich: personalization prevents a guard rule about fiscal year 2022 from being misapplied to a query about fiscal year 2024, but generic Findings and Cautions must first be present to be personalized. The total gain from A to E is 14.50 macro points, from 53.00 to 67.50. Taken together, the five strategies show that tools, experience, and annotation make complementary contributions.

Table 4: Memory strategy ablation. Checkmarks indicate which fields are injected into the memory block for each strategy. Macro is the arithmetic mean across the four benchmarks.

Strategy	Q	A	E	Annot	BizB	F-E	F-M	MME	Macro
A	✓				54	67	53	38	53.00
B	✓	✓			58	66	61	40	56.25
C	✓	✓	✓		66	71	64	47	62.00
D	✓	✓		✓	65	64	63	42	58.50
E	✓	✓	✓	✓	69	81	67	53	67.50

E Retrieval Stability Analysis

E.1 Experimental Design

We evaluate retrieval composition across 11 bank snapshots, from 0 to 1200 entries, by computing cosine similarity between all 11,379 test query embeddings and each bank entry at the operating threshold $\tau = 0.65$. The snapshots are constructed as cumulative prefixes drawn uniformly across six benchmarks: entries from `finacumen/memory/v8_unified/meta.json` are partitioned by `source.dataset`, shuffled with seed 42, and the first $S/6$ entries per benchmark are concatenated for each target size $S \in \{0, 120, 240, \ldots, 1200\}$. Each successive snapshot retains all entries from the previous size and appends 20 additional entries per benchmark, yielding a cumulative prefix of 120 entries per step. A small fraction of entries lack resolvable embeddings, yielding actual bank cardinalities slightly below the nominal target sizes. While we fix the primary operational threshold at $\tau = 0.65$ for cross-benchmark analysis, we also sweep $\tau \in [0.50, 0.90]$ to examine how retrieval density and volatility scale under different matching strictness.

E.2 Retrieval Metrics

We report four metrics, where $R_n(q)$ denotes the set of entries retrieved for query q at snapshot n and Q denotes the full query set of size 11,379.

$$H_n = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[|R_n(q)| > 0]$$

Hit rate H_n measures the fraction of queries receiving at least one retrieved entry at bank size n.

$$\bar{E}_n = \frac{1}{|Q|} \sum_{q \in Q} |R_n(q)|$$

Mean entries per query $\bar{E}_n$ captures retrieval density.

For adjacent snapshots n_1 and n_2, the per-query Jaccard similarity is defined with an explicit case for two empty retrieval sets:

$$J_q(n_1, n_2) = \begin{cases} 1, & \text{if } R_{n_1}(q) = R_{n_2}(q) = \emptyset, \\ \dfrac{|R_{n_1}(q) \cap R_{n_2}(q)|}{|R_{n_1}(q) \cup R_{n_2}(q)|}, & \text{otherwise.} \end{cases}$$

$$J_{n_1,n_2} = \frac{1}{|Q|} \sum_{q \in Q} J_q(n_1, n_2)$$

Adjacent-size Jaccard similarity J_{n_1,n_2} quantifies compositional stability across snapshots. Assigning $J_q = 1$ when both retrieval sets are empty inflates Jaccard estimates for benchmarks with low hit rates, a phenomenon of trivial stability discussed in Section E.4.

$$C_{n_1,n_2} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[R_{n_1}(q) \neq R_{n_2}(q)]$$

Change fraction C_{n_1,n_2} captures retrieval volatility as the strict proportion of queries whose retrieval set differs between snapshots.

Per-dataset hit rate and Jaccard curves, together with their global aggregates, appear in Figures 6 and 8. Table 5 summarizes the terminal-state metrics at bank size 1200 for $\tau = 0.65$.

E.3 Coverage and Retrieval Density

Hit rate saturates beyond 600 entries across all benchmarks: FinTMM reaches 90.4% at 1200 entries whereas FinMME reaches only 47.5%. This gap reflects divergent semantic affinity between benchmark query distributions and the patterns memorized in the bank. Chart-oriented FinMME queries activate substantially fewer stored experiences than numerical reasoning queries, consistent with a bank composition that over-represents text- and table-centric failure cases. Mean entries per query confirms this pattern: at $\tau = 0.65$, FinMMR-Medium and FinMMR-Hard retrieve 54.5 and 38.2

Table 5: Terminal retrieval state at bank size $N = 1200$ under $\tau = 0.65$. H_{1200} denotes hit rate, $\bar{E}_{1200}$ mean entries per query, and $J_{1080,1200}$ the Jaccard similarity at the final adjacent-size transition.

Benchmark	H_{1200}	$\bar{E}_{1200}$	$J_{1080,1200}$
BizBench	0.68	5.72	0.93
FinMMR-Easy	0.80	9.63	0.93
FinMMR-Medium	0.90	54.54	0.90
FinMMR-Hard	0.89	38.16	0.91
FinTMM	0.90	3.97	0.91
FinMME	0.48	1.15	0.95
Global	0.71	13.07	0.93

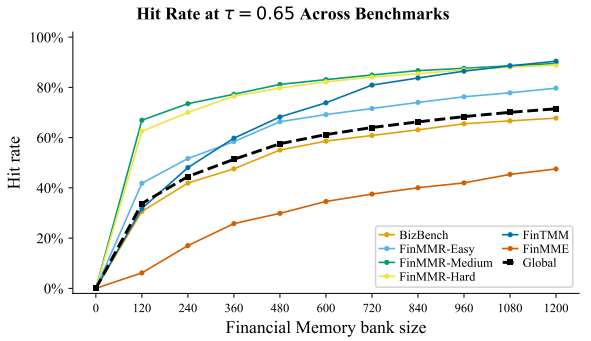


Figure 6: Hit rate as a function of FM bank size at $\tau = 0.65$, with a global aggregate curve. All benchmarks exhibit saturation beyond 600 entries; cross-dataset variation reflects divergent semantic affinity between benchmark query distributions and the bank's memorized patterns.

entries per query on average, while FinMME retrieves 1.1. FinTMM presents an intermediate profile with a hit rate of 90.4% but a mean entry count of only 4.0, indicating that its queries resolve with sparser yet highly targeted retrieval sets.

Under threshold-based retrieval, the expected number of retrieved entries follows $E[|R_n(q)|] = n \cdot p$, where $p = P(\text{sim}(q, e) \geq \tau)$ is independent of the current bank size. This produces the nearly linear growth of mean entries observed in Figure 7. Because new entries are purely additive under threshold filtering, the retrieval set monotonically expands: $R_n(q) \subseteq R_{n+\Delta n}(q)$ holds for all queries.

E.4 Retrieval Stability

Since $R_n(q) \subseteq R_{n+\Delta n}(q)$ holds under threshold retrieval, the expected Jaccard similarity between

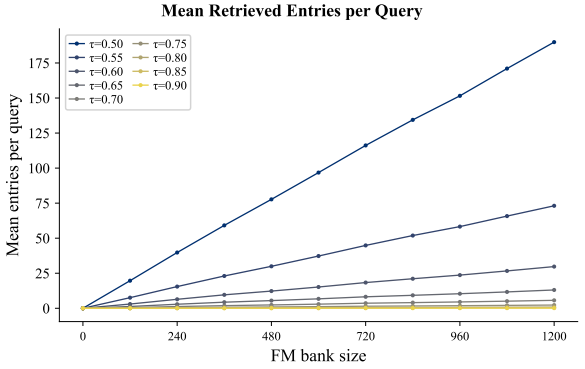


Figure 7: Mean number of retrieved entries per query aggregated across all 11,379 test queries. Each curve corresponds to one cosine threshold; the near-linear growth is predicted by $E[|R_n(q)|] = n \cdot p$.

adjacent snapshots has the closed form

$$E[J_{n,n+\Delta n}] \approx \frac{n \cdot p}{(n + \Delta n) \cdot p} = \frac{n}{n + \Delta n},$$

which increases monotonically and approaches 1.0 as n grows. This matches the dominant trend in Figure 8: all benchmarks exhibit a monotonic rise toward saturation, with the final transition values exceeding 0.90 for every dataset. Minor terminal fluctuations, such as the 0.004 drop observed in FinMMR-Medium, are consistent with sample variance: at specific transitions the empirical retrieval probability $\hat{p}$ of newly admitted entries deviates slightly from the population p, producing sub-thousandth-level oscillations that are not statistically significant.

The high Jaccard values of FinMME require careful interpretation. FinMME exhibits the lowest hit rate among all benchmarks, reaching only 47.5% at 1200 entries. For the remaining 52.5% of FinMME queries no entries are retrieved at either snapshot, and the convention $J_q(\emptyset, \emptyset) = 1$ applies. This trivial stability inflates the observed Jaccard toward 1.0 and partially accounts for FinMME's higher Jaccard curve. For benchmarks with sufficient retrieval coverage the Jaccard trajectory reflects genuine compositional convergence.

The change fraction C_{n_1,n_2} measures the proportion of queries for which at least one newly admitted entry exceeds the similarity threshold. Under threshold retrieval, a query's retrieval set changes if and only if at least one entry among the $\Delta n = 120$ newly added entries satisfies the similarity condition. This probability is

$$P(\text{Change}) = 1 - (1 - p)^{\Delta n},$$

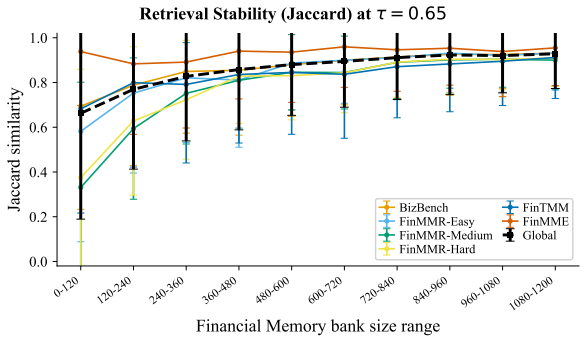


Figure 8: Adjacent-bank Jaccard similarity at $\tau = 0.65$ with error bars indicating one standard deviation across queries. Jaccard similarity increases monotonically toward 1.0, consistent with $E[J] \approx n/(n + \Delta n)$; minor local fluctuations are attributable to sample variance in newly added entries.

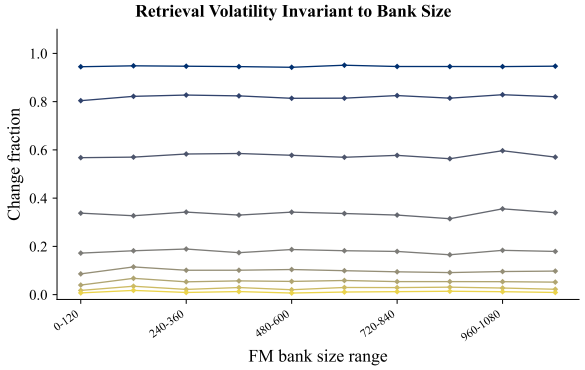


Figure 9: Change fraction across adjacent bank snapshots for nine cosine thresholds. The near-horizontal trajectory for each threshold confirms that retrieval volatility is invariant to bank size, as predicted by $P(\text{Change}) = 1 - (1 - p)^{\Delta n}$.

which depends solely on τ and the fixed step size Δn, not on the current bank size n. Empirically, the change fraction curves in Figure 9 are near-horizontal across all bank size transitions for every threshold value, confirming this invariance.

Taken together, these results establish that under threshold-based retrieval, retrieval stability is an inherent consequence of the retrieval mechanism rather than an emergent property of bank size. Jaccard similarity converges toward 1.0 as $n/(n+\Delta n)$ approaches unity, and change fraction remains constant at $1 - (1 - p)^{\Delta n}$ independent of n. For practitioners, adjusting the similarity threshold τ offers a more direct lever for controlling retrieval churn than limiting bank size.

F Benchmark Examples

G Experimental Details

This appendix documents the experimental protocol behind Table 1 and the ablation analyses. The goal is to make the comparison reproducible while keeping the main text focused on the central findings.

G.1 Evaluation Protocols

We evaluate FINACUMEN on the official test split of each benchmark and follow the native metric defined by the corresponding benchmark paper. For BizBench SEC-NUM, predictions are normalized by removing currency markers, percentage signs, commas, and whitespace before exact-match comparison with a 1% relative numerical tolerance. For FinMMR, we report accuracy on the Easy, Medium, and Hard splits using the official 0.2% relative tolerance after numeric and unit normalization. For FinTMMBench, we use the benchmark's GPT-4o-mini judge to decide whether the prediction and the reference answer express the same financial fact. For FinMME, we report Avg as the arithmetic mean of Compre, FG, Reason, Single, Multi, and Cal. The reconstruction and consistency check are provided below.

All reported FINACUMEN results follow these native scoring rules. We do not introduce a unified post-hoc metric across datasets because the benchmarks differ substantially in answer form, ranging from exact quantities to temporally grounded retrieval answers and chart-based multiple-choice questions.

Following Table 3 of FinMME (Luo et al., 2025), we compute Avg as the arithmetic mean of the six reported dimensions:

$$\text{Avg} = \frac{1}{6}\big(\text{Compre} + \text{FG} + \text{Reason} + \text{Single} + \text{Multi} + \text{Cal}\big).$$

Applying the published dimension scores reproduces the reported Avg values within rounding.

The released annotations do not provide official per-sample domain labels. We therefore reconstruct the six-dimension partition, and the resulting cognitive split differs from Table 2 of FinMME by at most 0.02 percentage points.

G.2 FinAcumen Configuration

The backbone for all FINACUMEN and Base rows is Qwen3-VL-8B-Instruct. The backbone parame-

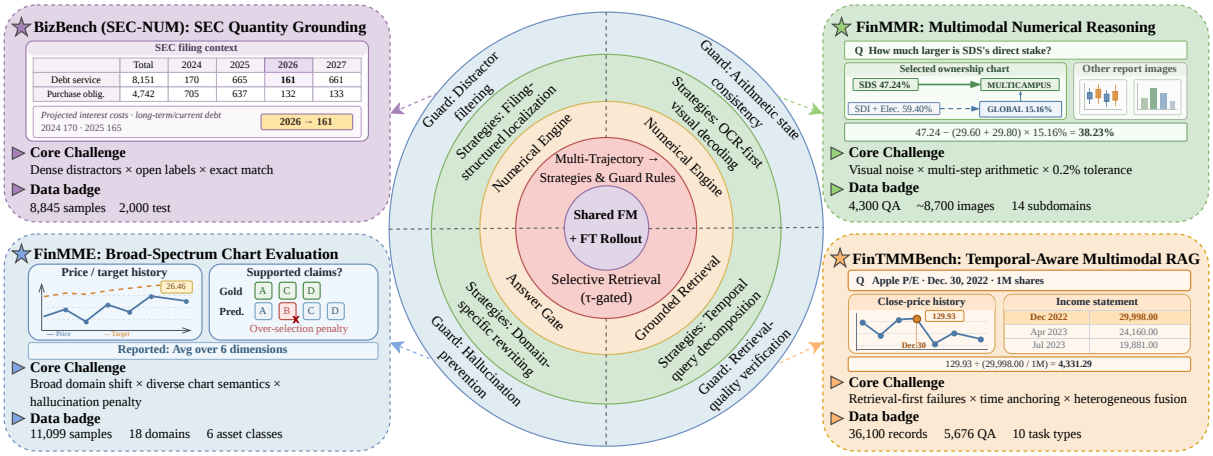


Figure 10: Example items from the four financial multimodal benchmarks evaluated in this work.

Table 6: Benchmark settings used in the main experiments. Each benchmark keeps its original split and native scoring rule.

Benchmark	Setting	Test size	Primary challenge	Reported metric
BizBench SEC-NUM	SEC filings	2000	Numeric grounding under distractors	Accuracy, 1% tolerance
FinMMR	Easy / Medium / Hard	1200 / 1200 / 1000	Multi-image financial reasoning	Accuracy, 0.2% tolerance
FinTMMBench	Temporal multimodal QA	1136	Retrieval over tables, news, prices, and charts	GPT-4o-mini judge accuracy
FinMME	Chart-intensive financial QA	2220	Chart understanding across financial domains	Avg

ters are frozen throughout; FINACUMEN changes only the inference-time context and tool execution path through FM and FT. Unless otherwise stated, decoding uses temperature 0 for final test evaluation, and the ReAct-style solver is capped at 16 tool-interaction steps. The tool suite is fixed across all FINACUMEN runs and consists of Python execution for deterministic arithmetic, financial data lookup, OCR-based visual structure extraction, and a termination gate that checks source traceability, unit consistency, and answer format.

For each source instance, memory construction samples $K = 8$ independent trajectories before scoring and consolidation. The memory bank is built before final testing from training/validation trajectories and is kept read-only during evaluation. Test questions never write new entries into the bank. Each memory entry stores the source question, gold answer, distilled findings from successful trajectories, and cautions extracted from failed trajectories. At inference time, the query embedding is formed from the question and the first 600 characters of the context, then compared against pre-computed memory-entry embeddings. Entries below the cosine threshold $\tau = 0.65$ are discarded, duplicate source items are removed, and at most $k_{\max} = 5$ retrieved entries are rendered into the memory prefix for the main results in Table 1. The Base row disables both FM and FT, while ablations keep the backbone, decoding configuration, tool interfaces, and benchmark scoring fixed. All other retrieval and tool parameters follow the architecture described in Section 4.

G.3 Baseline Attribution

The non-FINACUMEN baseline numbers in Table 1 are taken from the original benchmark papers rather than re-run in our environment: BizBench results from Krumdick et al. (2024), FinMMR results from Tang et al. (2025), FinTMMBench results from Zhu et al. (2025), and FinMME results from Luo et al. (2025). This choice follows the standard leaderboard-comparison protocol for these benchmarks and avoids conflating our framework contribution with differences in closed-source API versions, prompting details, or evaluation access. When a benchmark paper reports a model family rather than a single shared variant across all columns, Table 1 groups the results by family and the per-cell model variant is specified below.

Table 7: Consistency check for the FinMME Avg metric.

Model	Recomputed	Published	Difference
Gemini 2.0 Flash	51.8500	51.85	0.000
Claude 3.5 Sonnet	48.1950	48.20	0.005
GPT-4o	46.5583	46.56	0.002
Qwen2.5-VL-72B	52.5417	52.54	0.002

Table 8: Per-cell model variants for grouped General LLM baselines in Table 1. All numbers are copied from the corresponding benchmark papers. A dash indicates that the source benchmark did not report the model.

Model family	BizBench	FinMMR-Easy	FinMMR-Medium	FinMMR-Hard	FinTMMBench	FinMME
GPT	GPT-4	GPT-4o	GPT-4o	GPT-4o	GPT-4o-mini	GPT-4o
Claude Sonnet	Claude 3.7 Sonnet	Claude 3.7 Sonnet	Claude 3.7 Sonnet	Claude 3.7 Sonnet	–	Claude 3.5 Sonnet
Llama	Llama 2 70B	Llama 4 Maverick 17B	Llama 4 Maverick 17B	Llama 4 Maverick 17B	Llama 3.2 11B	–

For Qwen2.5-VL-72B, Gemini 2.0 Flash, and the financial-domain baselines, we preserve the exact variants and scores reported by the benchmark sources whenever available. Missing cells in Table 1 mean that the benchmark paper did not report that model under the official setting, not that the model failed evaluation.

H Repeated-Run Uncertainty

We report measurement uncertainty for Base and FINACUMEN from five independent runs under identical conditions. Each cell contains the mean, empirical standard deviation, and 95% confidence interval. The interval is computed as

$$\bar{x} \pm t_{0.975,4} \frac{s}{\sqrt{5}}, \qquad t_{0.975,4} = 2.776.$$

The reported intervals are separated in each evaluated setting. FINACUMEN has lower empirical variance on FinMMR Easy and FinMMR Medium and higher empirical variance on the remaining settings.

I Experience-Memory Baselines

We compare FINACUMEN with Reflexion (Shinn et al., 2023) and ExpeL (Zhao et al., 2024). The three methods use Qwen3-VL-8B-Instruct as the backbone, share the same tool environment and training data for experience construction, and freeze their memory banks before evaluation while retaining their respective experience representations. This protocol focuses the comparison on how experience is represented and used.

The four comparisons favor FINACUMEN under this protocol. This pattern may reflect the separation of successful strategies and failure patterns into Findings and Cautions, together with experience activation conditioned on sufficient relevance. Appendix D provides complementary component evidence. With Q, A, Annot, and the tool configuration unchanged, adding the Experience field increases macro accuracy from 58.50 to 67.50.

J Cross-Backbone and Scale Generalization

J.1 Additional Backbones

We evaluate three additional backbones on BizBench under the configuration used for Table 1.

Table 11: BizBench results with additional backbones.

Base model	Base	FINACUMEN
GLM-4.6V	30.40	75.58
GPT-4o-mini	23.60	59.60
Gemini-2.5-Flash-Lite	32.60	48.40

The observed differences are positive for the three evaluated backbones, providing evidence that the performance gain is not confined to Qwen3-VL-8B-Instruct.

J.2 Larger Backbone

We further evaluate Qwen3-VL-235B-A22B-Instruct on the 2000-item BizBench SEC-NUM test set. The prompt, tool interfaces, and decoding configuration remain fixed across the three settings.

Table 9: Repeated-run uncertainty for Base and FINACUMEN across five independent runs. Each cell reports the mean, standard deviation, and 95% confidence interval.

Benchmark	System	Mean ± std	95% CI
BizBench	Base	27.67 ± 1.00	$[26.43, 28.91]$
	FINACUMEN	68.65 ± 2.45	$[65.61, 71.69]$
FinMMR Easy	Base	59.17 ± 1.64	$[57.13, 61.21]$
	FINACUMEN	81.67 ± 1.14	$[80.25, 83.09]$
FinMMR Medium	Base	45.75 ± 1.30	$[44.14, 47.36]$
	FINACUMEN	67.00 ± 0.71	$[66.12, 67.88]$
FinMMR Hard	Base	31.40 ± 1.00	$[30.16, 32.64]$
	FINACUMEN	46.10 ± 2.86	$[42.55, 49.65]$
FinTMMBench	Base	16.20 ± 0.55	$[15.52, 16.88]$
	FINACUMEN	19.98 ± 0.89	$[18.88, 21.08]$
FinMME	Base	27.84 ± 0.84	$[26.80, 28.88]$
	FINACUMEN	51.08 ± 2.24	$[48.30, 53.86]$

Table 10: Comparison with experience-memory baselines under a shared backbone and tool environment.

Setting	Method	Accuracy
FinMMR E	Reflexion	77.17
	ExpeL	77.17
	FINACUMEN	81.67
FinMMR M	Reflexion	63.58
	ExpeL	60.58
	FINACUMEN	67.00
FinTMM	Reflexion	14.52
	ExpeL	16.73
	FINACUMEN	19.98
FinMME	Reflexion	42.97
	ExpeL	45.77
	FINACUMEN	51.08

Table 12: Component results with Qwen3-VL-235B-A22B-Instruct on BizBench SEC-NUM.

Setting	Accuracy	Δ Base	Δ FT only
Base	57.45	–	–
FT only	69.30	11.85	–
FINACUMEN	75.10	17.65	5.80

FT only improves accuracy by 11.85 points over Base, and Financial Memory adds 5.80 points over FT only. The result is consistent with the complementary pattern observed in the 8B ablation.

K Cost Analysis

K.1 Offline Construction Cost

FINACUMEN samples $K = 8$ independent trajectories per source instance. Based on measurements from six source instances, we estimate the offline cost of constructing the approximately 1800-entry memory bank in Table 13. The sequential time was measured on an Intel Core i7-12700H processor. The bank is constructed and frozen before evaluation, so trajectory sampling, scoring, and consolidation are not repeated during inference.

Table 13: Estimated offline memory construction cost for the approximately 1800-entry bank.

Measure	Estimate
API calls	64,200
Tokens	244.43 million
Cost	CNY 142.97
Sequential time	78.12 hours
Storage	8.19 MiB

K.2 Inference Cost

We sample one sixth of the test instances from each of the six evaluation settings and conduct paired runs with and without the bank. Table 14 reports the average inference cost per question.

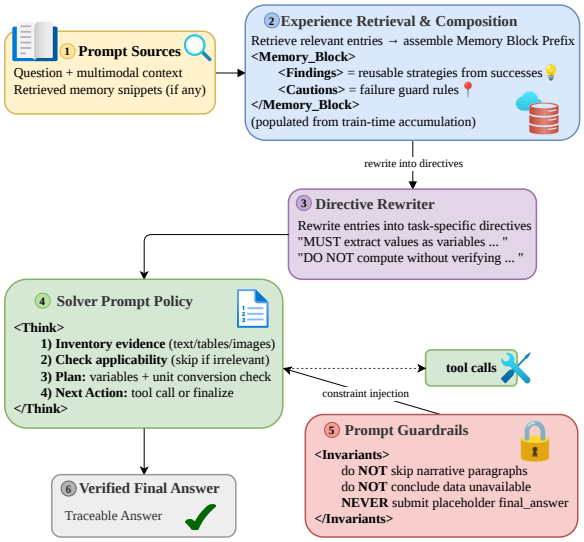


Figure 11: Prompt orchestration in FINACUMEN.

Table 14: Average inference cost per question in paired runs.

Measure	With bank	Without bank
Input tokens	40,462.02	19,072.23
Output tokens	2,061.11	971.28
Total tokens	42,523.13	20,043.51
Input cost	CNY 0.0202	CNY 0.0095
Output cost	CNY 0.0041	CNY 0.0019
Total cost	CNY 0.0244	CNY 0.0115

Memory activation increases the average total token count from 20,043.51 to 42,523.13 and the average cost from CNY 0.0115 to CNY 0.0244 per question. The approximately 1800-entry bank is constructed once and shared across queries. Selective retrieval bypasses annotation and memory injection when no experience satisfies the retrieval condition.

L Prompt Templates

Figure 11 provides a visual overview of the prompt orchestration in FINACUMEN. Each prompt excerpt is labeled by its corresponding agent role, described in §4.3.

Solver agent (FinAcumen System Prompt, excerpted)

```
<Memory_Handling>
The input may contain a <Memory_Block>
    before the <Problem> block.
Each <Entry> has:
- <Question>: The original problem that
    produced this experience.
  Use it to judge whether your current
    problem is in a similar scenario
  (industry, data source, problem shape).
- <Answer>: The correct answer to that
    original problem.
  Use as a format reference --- not to
    copy the value, but to understand
  what kind of output your problem
    expects (number, text, percentage,
    etc.).
- <Experience>: Distilled reusable rules
    organized as
  <Findings> and <Cautions>:
    <Findings>: Patterns that succeeded
    --- follow these strategies.
    <Cautions>: Guard rules from failures
     --- apply if the condition triggers.

The block ends with an <OptOut> --- use
    it only when an entry truly
does not fit.
</Memory_Handling>

<Think_Steps>
<Think>
### 1. Input Inventory (MANDATORY --- do
    NOT skip to computation)
Output a complete inventory before any
    tool call.
[NARRATIVE] Every standalone number with
    its unit and context, one per line.
[TABLES] ALL row labels with ALL column
    values; list each cell.
[IMAGES] ALL fields from OCR/vision; flag
     ambiguous readings.
[TERM_MATCH] Map question terms to
    inventory entries.
  If a term has NO direct match, list
    candidate matches explicitly.

If any section is incomplete, you have
    not finished reading.
DO NOT proceed to Strategy until the
    inventory covers ALL of Context.

### 2. Experience Applicability (if
    Memory_Block present)
{step2_text}

### 3. Problem Understanding
- What quantity? Unit? Precision? Check
    Instruction.
- Resolve TERM_MATCH warnings.
- Output TYPE: ENTITY NAME, NUMERIC VALUE,
     DESCRIPTIVE SENTENCE,
  VERDICT, or NEWS SENTIMENT. Match
    answer format to expected type:
  * ENTITY -> terminate(final_answer="
    Company Name")
  * NUMERIC -> terminate(final_answer="
    VALUE UNIT")
  * DESCRIPTIVE -> terminate(final_answer
    ="Direction: description with numbers
    ")
  * VERDICT -> terminate(final_answer="
    Yes/No" or "true/false")
```

```
  * NEWS -> terminate(final_answer="Yes"
    or "No")

### 4. Strategy
Write ALL values as python variables
    before computing
(e.g., rev_2022 = 125.4).
Check unit conversion (millions<>billions,
     %<>decimal).

### 5. Next Action
Tool call or terminate.
</Think>
After </Think>, emit the tool call. No &
    lt;Answer&gt; block.
</Think_Steps>

<Invariants>
- NEVER skip narrative paragraphs to jump
     to tables.
  Context is one document --- prose AND
    tables must both be read.
- "data not available" is never true. If
    you think data is missing,
  re-read from the beginning --- you
    missed a section.
- NEVER submit placeholder text as
    final_answer: avoid
  'I will terminate', 'data not available
    ', 'insufficient data',
  empty strings.
- final_answer MUST contain the actual
    answer value, not a meta-status.
</Invariants>
```

Summary agent—Experience synthesis (excerpted)

```
<findings_guide>
Write generalized reusable strategies.
    Strip ALL specific values:
numbers, years, entity names.
Format: "To <goal>, use/do <method>."
Trigger: what information format triggers
     this? What question type?
Method: what to do step by step? Which
    tool? Which formula?
Each finding is one sentence, self-
    contained and executable.

<cautions_guide>
Write generalized guard rules. Strip ALL
    specific values.
Format: "When <detectable condition>, <
    corrective action>."
Condition must be detectable from
    question/context text
(not hindsight).
Action must be concrete: what to check,
    how to check, what to do.

<Output>
Return ONLY:
<Think>
analysis: [problem-specific derivation]
-> From above, generalize findings:
-> From above, generalize cautions:
-> Verify: does each item have trace
    support?
   Is the condition detectable?
</Think>
<Answer>
{"findings": ["To ..., use/do ..."],
 "cautions":  ["When ..., ..."],
 "analysis":  "<<
    REPLACE_WITH_YOUR_PROBLEM_DERIVATION
    >>"}
</Answer>
```

Annotator agent—Directive annotation (excerpted)

```
<Task>
For each entry write 1-3 directive
    sentences using MUST / DO NOT.
Extract the entry's methodology (
    extraction pattern, computation logic
    ,
verification step) and adapt it to the
    current problem.
NEVER write an empty annotation.
Set "useful": false ONLY when the task
    TYPE is fundamentally different
(chart reading vs. table lookup vs.
    narrative extraction).
Otherwise, useful MUST be true.

<Generalization_Rule>
Map the memory's METHODOLOGY, not its
    literal entity names.
Memory: specific years/entity names ->
    current problem: relevant equivalents
    .
If you cannot map entities, the entry is
    NOT useful.

<Output>
[{"i": 0, "useful": true,
  "annotation": "MUST extract values from
     Context table as python
   variables before computing. DO NOT
    compute without verifying which
   table row matches the question."},
    ...]
```

Memory_Block XML injection format

```
<Memory_Block>
  <Field_Guide>
  Each Entry is a past experience:
    - Question: the original problem (use
      to judge scenario similarity)
    - Answer: the correct answer (format
    reference only, never copy value)
    - Experience: distilled reusable
    rules organized as:
      - Findings: patterns that succeeded
      -> follow these strategies
```

```
      - Cautions: guard rules from
     failures -> apply if condition
     triggers
  </Field_Guide>
  <Entry>
    <Question>original problem text</
    Question>
    <Answer>correct answer</Answer>
    <Experience>
      <Findings>reusable strategies from
    successes</Findings>
      <Cautions>guard rules from failures
    </Cautions>
    </Experience>
  </Entry>
  <OptOut>If any entry above does NOT
     actually fit this problem,
    ignore it.</OptOut>
</Memory_Block>
```